Security Audit - Marinade Stake Rent Reduction Fix

conducted by Neodyme AG

Lead Auditor:	Jasper Slusallek
Administrative Lead:	Jasper Slusallek

October 02, 2026

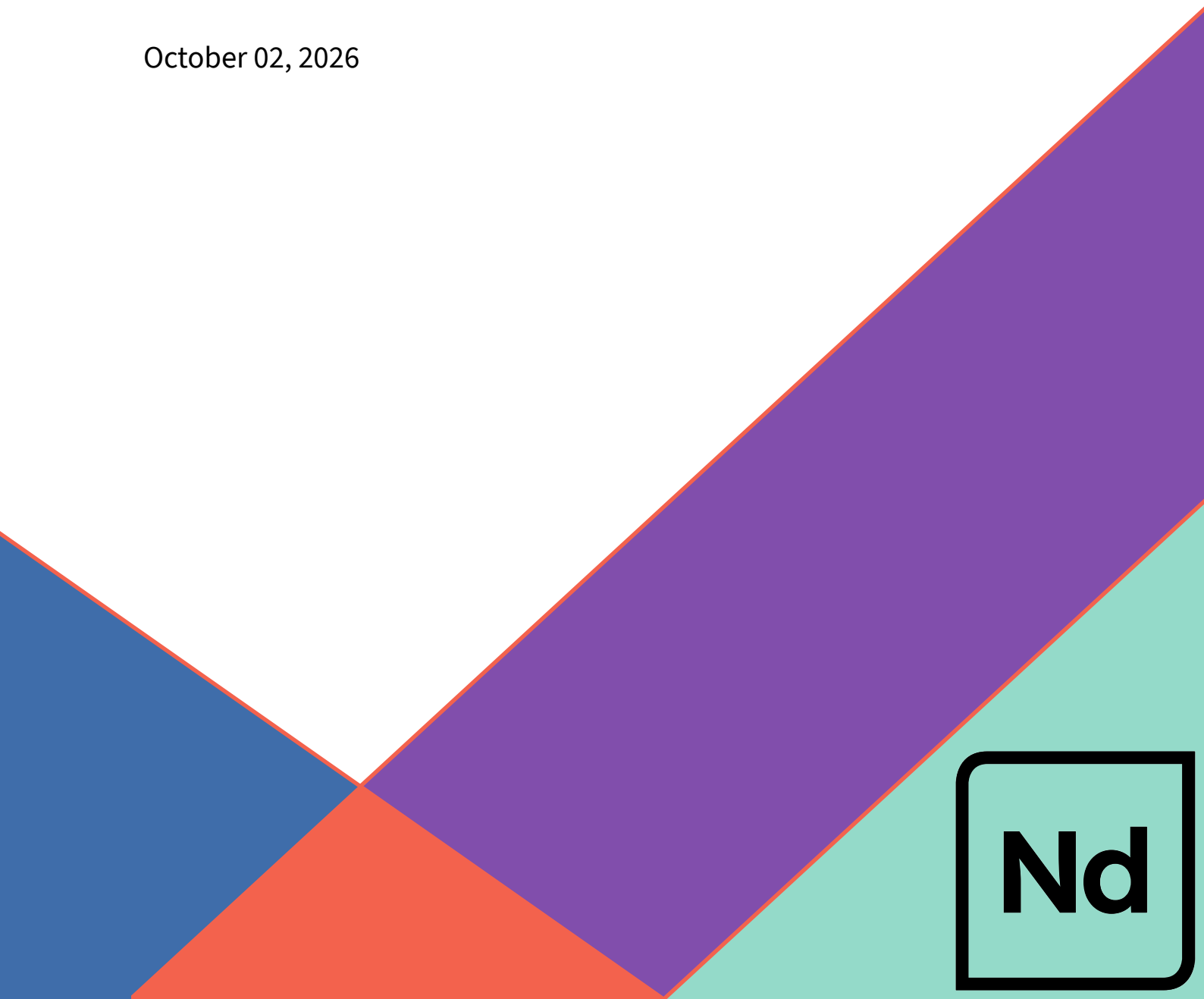


Table of Contents

1 Executive Summary **3**

2 Introduction **4**
Summary of Findings 4

3 Scope **5**

4 Changes **6**
Rent Reduction Fix 6

5 Findings **7**

6 Comments **8**

Appendices

A About Neodyme **9**

B Methodology **10**
Select Common Vulnerabilities 10

C Vulnerability Severity Rating **12**

1 | Executive Summary

Neodyme reviewed a very small change to **Marinade's** on-chain liquid staking program in September 2026. The change adapts the program to the reduction of Solana's rent-exempt minimum by computing stake account rent from the Rent sysvar instead of the frozen `rent_exempt_reserve` field of stake accounts.

The threat modeling and economic analysis of the original 2023 audit remain valid.

According to Neodyme's Rating Classification, **0 security relevant** and **0 informational** issues were found. The number of findings, grouped by severity, can be seen in Figure 1.

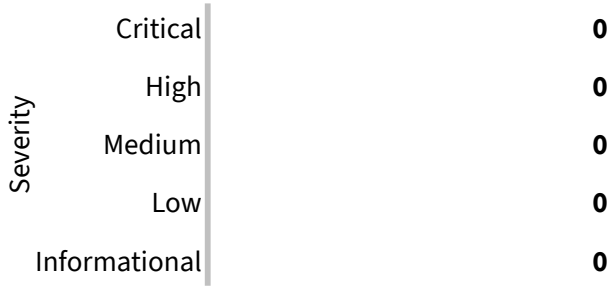


Figure 1: Overview of Findings

Neodyme shared a few comments with the Marinade team, listed in Section 6.

2 | Introduction

In September 2026, Marinade engaged Neodyme to review the rent reduction fix to their on-chain liquid staking program. The lead auditor was the same as in the previous audits of the program and has extensive knowledge of Solana's stake program, which Marinade interacts with.

The review focused on the impact of the changes on the contract's technical security.

Summary of Findings

In total, the review revealed:

0 critical • **0** high-severity • **0** medium-severity • **0** low-severity • **0** informational

issues.

3 | Scope

Neodyme considers the changes to the source code, located at https://github.com/marinade-finance/liquid-staking-program, in scope for this review. Third-party dependencies are not in scope.

The relevant source code revisions are:

- `b6965daef808b2712259d000d46175858e11f8a0` · First reviewed commit
- `d4e20237f08add67f030927fe12a097b62b837f5` · Last reviewed commit

4 | Changes

Following the gradual decrease of rent on Solana, and since the stake program no longer updates the `rent_exempt_reserve` field in a stake account's `Meta`, the program logic needed to be updated to not rely on the latter. The changes replace all uses of the field.

Rent Reduction Fix

Commit `b6965da` adds a helper `stake_rent_exempt_reserve()`, which computes the rent-exempt minimum of a stake account from the Rent sysvar. It replaces `meta.rent_exempt_reserve` in:

- `CreateCanonicalStake`: the source balance check is changed from equality to a lower bound, as there may be excess rent due to the rent decreases. This excess rent was forbidden before, but is now allowed.
- `UpdateActive` and `UpdateDeactivated`: both subtract the rent from the stake account balance to find rewards or extra lamports. They now subtract the live rent instead of the `rent_exempt_reserve` field. `UpdateDeactivated` also refunds the live rent to the operational SOL account.
- `DepositStakeAccount`: The balance of a deposited stake account must now be at least the delegation plus the live rent, and at most the delegation plus the old pre-decrease rent of 2,282,880 lamports. This is to allow for any stake account to be deposited that has allowable rent, where allowable means any rent amount above the current minimum and below the old pre-decrease rent amount.

Additionally, `DepositStakeAccount` now asserts the deposited account's data length to be equal to the size of `StakeState`, since the rent is computed from that size. A new error `WrongStakeAccountDataLength` is returned otherwise.

5 | Findings

This section normally outlines all of our findings, but this review had none.

6 | Comments

Neodyme shared the following comments with the Marinade team:

- **Rent refunded to the operational wallet.** After the change, stake accounts the operational wallet paid rent for are only refunded the current, lower rent. The difference stays in the pool and is treated as MEV rewards. With around 300 stake accounts and around 0.002 SOL less rent per account, the operational wallet will lose around 0.6 SOL in total once the rent reduction schedule is complete. We assume this is acceptable for Marinade.
- **Use `data_len()` in `stake_rent_exempt_reserve()`.** This helper function computes the rent from `size_of::<StakeState>()`. This gives the correct result today, and Marinade's pool contains no legacy stake accounts with a different size. However, using the account's `data_len()` instead matches what the stake program does and stays correct should the stake account layout change in the future (though this is unlikely).
- **Rent increases.** The changes assume that rent will never be increased again, for example by SIMD-0438. If it is increased, further changes to the program will be needed.

A | About Neodyme

Security is difficult.

To understand and break complex things, you need a certain type of people. People who thrive in complexity, who love to play around with code, and who don't stop exploring until they fully understand every aspect of it. That's us.

Our team never outsources audits. Having found over 80 High or Critical bugs in Solana's core code itself, we believe that Neodyme hosts the most qualified auditors for Solana programs. We've also found and disclosed critical vulnerabilities in many of Solana's top projects and have responsibly disclosed issues that could have resulted in the theft of over $10B in TVL on the Solana blockchain.

All of our team members have a background in competitive hacking. During such hacking competitions, called CTFs, we competed and collaborated while finding vulnerabilities, breaking encryption, reverse engineering complicated algorithms, and much more. Through the years, many of our team members have won national and international hacking competitions, and keep ranking highly among some of the hardest CTF events worldwide. In 2020, some of our members started experimenting with validators and became active members in the early Solana community. With the prospect of an interesting technical challenge and bug bounties, they quickly encouraged others from our CTF team to look for security issues in Solana. The result was so successful that after reporting several bugs, in 2021, the Solana Foundation contracted us for source code auditing. As a result, Neodyme was born.



B | Methodology

We are not checklist auditors.

In fact, we pride ourselves on that. We adapt our approach to each audit, investing considerable time into understanding the program upfront and exploring its expected behavior, edge cases, invariants, and ways in which the latter could be violated.

We use our uniquely deep knowledge of Solana internals, and our years-long experience in auditing Solana programs to find bugs that others miss. We often extend our audit to cover off-chain components in order to see how users could be tricked or the contract affected by bugs in those components.

Nonetheless, we also have a list of common vulnerability classes, which we always exhaustively look for. We provide a sample of this list here.

Select Common Vulnerabilities

Our most common findings are still specific to Solana itself. Among these are vulnerabilities such as the ones listed below:

- Insufficient validation, such as:
  - Missing ownership checks
  - Missing signer checks
  - Signed invocation of unverified programs
  - Account confusions
  - Missing freeze authority checks
  - Insufficient SPL account verification
  - Dangerous user-controlled bumps
  - Insufficient Anchor account linkage
- Account reinitialization vulnerabilities
- Account creation DoS
- Redeployment with cross-instance confusion
- Missing rent exemption assertion
- Casting truncation
- Arithmetic over- or underflows
- Numerical precision and rounding errors
- Anchor pitfalls, such as accounts not being reloaded
- Non-unique seeds
- Issues arising from CPI recursion
- Log truncation vulnerabilities
- Vulnerabilities specific to integration of Token Extensions, for example unexpected external token hook calls

Apart from such Solana-specific findings, some of the most common vulnerabilities relate to the general logical structure of the contract. Specifically, such findings may be:

- Errors in business logic
- Mismatches between contract logic and project specifications
- General denial-of-service attacks
- Sybil attacks
- Incorrect usage of on-chain randomness
- Contract-specific low-level vulnerabilities, such as incorrect account memory management
- Vulnerability to economic attacks
- Allowing front-running or sandwiching attacks

Miscellaneous other findings are also routinely checked for, among them:

- Unsafe design decisions that might lead to vulnerabilities being introduced in the future
    - Additionally, any findings related to code consistency and cleanliness
- Rug pull mechanisms or hidden backdoors

Often, we also examine the authority structure of a contract, investigating their security as well as the impact on contract operations should they be compromised.

Over the years, we have found hundreds of high and critical severity findings, many of which are highly nontrivial and do not fall into the strict categories above. This is why our approach has always gone way beyond simply checking for common vulnerabilities. We believe that the only way to truly secure a program is a deep and tailored exploration that covers all aspects of a program, from small low-level bugs to complex logical vulnerabilities.

C | Vulnerability Severity Rating

We use the following guideline to classify the severity of vulnerabilities. Note that we assess each vulnerability on an individual basis and may deviate from these guidelines in cases where it is well-founded. In such cases, we always provide an explanation.

Severity	Description
CRITICAL	Vulnerabilities that will likely cause loss of funds. An attacker can trigger them with little or no preparation, or they are expected to happen accidentally. Effects are difficult to undo after they are detected.
HIGH	Bugs that can be used to set up loss of funds in a more limited capacity, or to render the contract unusable.
MEDIUM	Bugs that do not cause direct loss of funds but that may lead to other exploitable mechanisms, or that could be exploited to render the contract partially unusable.
LOW	Bugs that do not have a significant immediate impact and could be fixed easily after detection.
INFORMATIONAL	Bugs or inconsistencies that have little to no security impact, but are still noteworthy.

Additionally, we often provide the client with a list of nit-picks, i.e. findings whose severity lies below Informational. In general, these findings are not part of the report.

Neodyme AG

Dirnismaning 55
Halle 13
85748 Garching
Germany

E-Mail: contact@neodyme.io

https://neodyme.io